

TravelMate智能旅游助手系统数据库设计说明书

1 引言

1.1 编写目的

本文档旨在定义TravelMate系统的数据库结构、数据模型、存储策略及访问机制，为数据库开发、测试与维护提供依据，确保数据一致性、完整性及安全性。

1.2 项目概述

TravelMate是基于对话式人工智能的旅行规划助手，面向学生及年轻上班族，提供智能行程规划、预算估算、地图可视化等功能。系统采用微信小程序 + Python/FastAPI后端 + MySQL/PostgreSQL数据库架构。

1.3 术语与缩写

缩写/术语	英文全称/中文含义
DB	Database, 数据库
ERD	Entity-Relationship Diagram, 实体关系图
PK	Primary Key, 主键
FK	Foreign Key, 外键
POI	Point of Interest, 兴趣点
JSON	JavaScript Object Notation, 数据交换格式
LLM	Large Language Model, 大语言模型
API	Application Programming Interface, 应用程序编程接口

1.4 参考资料

- 《TravelMate系统设计说明书》
- GB/T 8567-2006 《计算机软件文档编制规范》
- MySQL 8.0 / PostgreSQL 14 官方文档

2 数据库环境说明

- 数据库管理系统**: MySQL 8.0 或 PostgreSQL 14
- 字符集**: UTF-8
- 事务支持**: ACID, 支持事务隔离级别
- 备份工具**: mysqldump / pg_dump, 支持定时备份
- 访问方式**: Python SQLAlchemy ORM + 原生SQL (必要时)

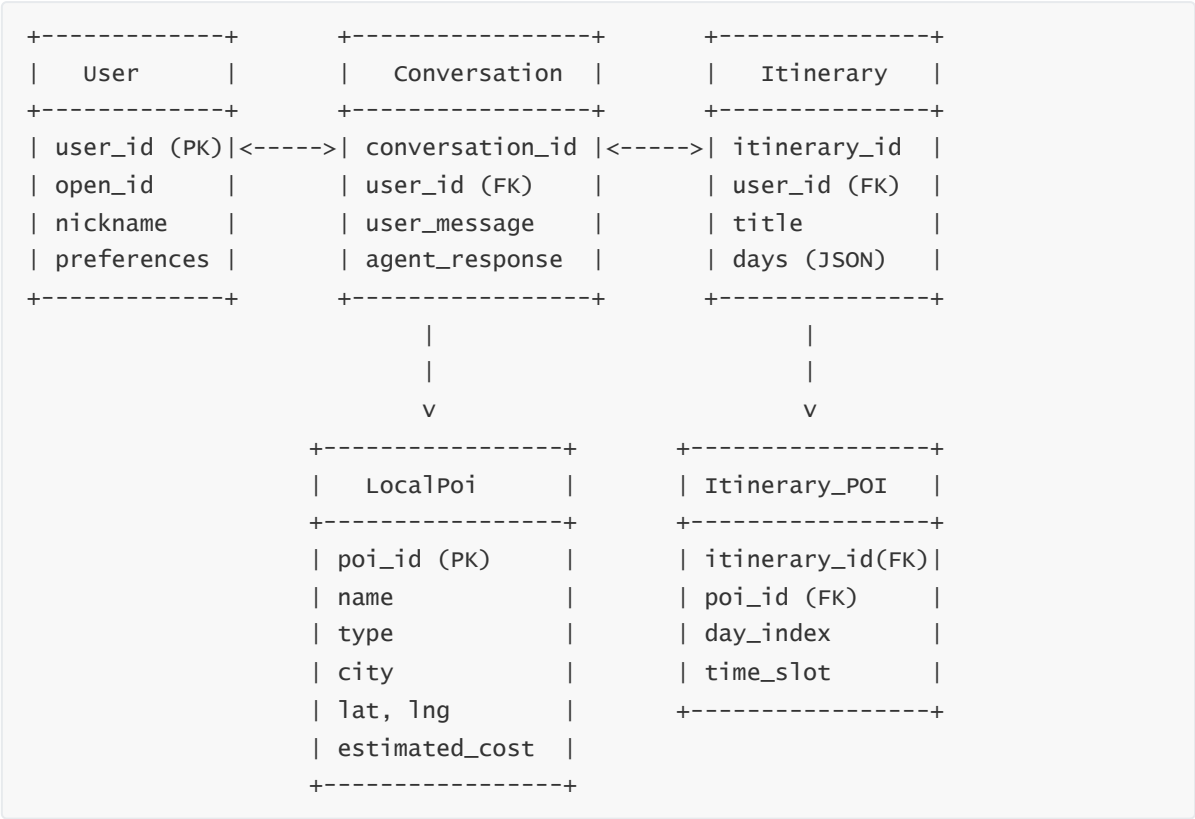
3 数据库命名规范

- 表名：英文名词复数，如 `users`、`itineraries`
- 字段名：小写蛇形命名，如 `user_id`、`created_at`
- 主键：表名_id，如 `user_id`、`itinerary_id`
- 外键：引用表名_id，如 `user_id`、`conversation_id`
- 索引：idx_表名_字段名，如 `idx_users_open_id`

4 数据库逻辑设计

4.1 概念模型（ERD）

实体关系图如下：



4.2 实体与属性说明

4.2.1 User（用户）

- 描述：存储用户基本信息和旅行偏好
- 属性：
 - `user_id`：主键，唯一标识
 - `open_id`：微信开放ID，唯一
 - `nickname`：用户昵称
 - `avatar_url`：头像URL
 - `travel_preferences`：JSON格式存储偏好设置

- `created_at`、`updated_at`：时间戳

4.2.2 Conversation（会话）

- **描述：**存储用户与系统的对话记录
- **属性：**
 - `conversation_id`：主键
 - `user_id`：外键，关联User
 - `user_message`：用户输入文本
 - `agent_response`：系统回复文本
 - `context_snapshot`：JSON格式上下文快照
 - `timestamp`：消息时间戳

4.2.3 Itinerary（行程）

- **描述：**存储用户生成的行程方案
- **属性：**
 - `itinerary_id`：主键
 - `user_id`：外键，关联User
 - `title`：行程标题
 - `days`：JSON格式存储每日活动安排
 - `total_budget`：总预算
 - `currency`：货币类型，默认CNY
 - `created_at`、`updated_at`：时间戳

4.2.4 LocalPoi（本地兴趣点）

- **描述：**存储景点、餐厅、酒店等POI数据
- **属性：**
 - `poi_id`：主键
 - `name`：POI名称
 - `type`：类型（景点、餐饮、酒店等）
 - `city`：所在城市
 - `lat`、`lng`：经纬度坐标
 - `estimated_cost`：估计费用
 - `tags`：JSON格式标签（如["人文","博物馆"]）
 - `source`：数据来源
 - `updated_at`：最后更新时间

4.2.5 Itinerary_POI（行程-POI关联）

- 描述：多对多关联表，记录行程中使用的POI及时间安排
- 属性：
 - itinerary_id：外键，关联Itinerary
 - poi_id：外键，关联LocalPoi
 - day_index：第几天
 - time_slot：时间段，如“09:00”

5 数据库物理设计

5.1 表结构定义

5.1.1 users

字段名	类型	约束	说明
user_id	VARCHAR(64)	PRIMARY KEY	用户ID
open_id	VARCHAR(128)	UNIQUE NOT NULL	微信 OpenID
nickname	VARCHAR(64)		昵称
avatar_url	TEXT		头像URL
travel_preferences	JSON		旅行偏好
created_at	DATETIME	DEFAULT NOW()	创建时间
updated_at	DATETIME	DEFAULT NOW() ON UPDATE NOW()	更新时间

5.1.2 conversations

字段名	类型	约束	说明
conversation_id	VARCHAR(64)	PRIMARY KEY	会话ID
user_id	VARCHAR(64)	FOREIGN KEY REFERENCES users(user_id)	用户ID
user_message	TEXT	NOT NULL	用户消息
agent_response	TEXT	NOT NULL	系统回复
context_snapshot	JSON		上下文快照
timestamp	DATETIME	DEFAULT NOW()	消息时间

5.1.3 itineraries

字段名	类型	约束	说明
itinerary_id	VARCHAR(64)	PRIMARY KEY	行程ID
user_id	VARCHAR(64)	FOREIGN KEY REFERENCES users(user_id)	用户ID
title	VARCHAR(128)	NOT NULL	行程标题
days	JSON	NOT NULL	行程天数与活动
total_budget	DECIMAL(10,2)		总预算
currency	VARCHAR(10)	DEFAULT 'CNY'	货币类型
created_at	DATETIME	DEFAULT NOW()	创建时间
updated_at	DATETIME	DEFAULT NOW() ON UPDATE NOW()	更新时间

5.1.4 local_pois

字段名	类型	约束	说明
poi_id	VARCHAR(64)	PRIMARY KEY	POI ID
name	VARCHAR(128)	NOT NULL	名称
type	VARCHAR(32)	NOT NULL	类型
city	VARCHAR(64)	NOT NULL	城市
lat	DECIMAL(10,6)		纬度
lng	DECIMAL(10,6)		经度
estimated_cost	DECIMAL(10,2)		估计费用
tags	JSON		标签
source	VARCHAR(64)		数据来源
updated_at	DATETIME	DEFAULT NOW() ON UPDATE NOW()	更新时间

5.1.5 itinerary_poi

字段名	类型	约束	说明
itinerary_id	VARCHAR(64)	FOREIGN KEY REFERENCES itineraries(itinerary_id)	行程ID
poi_id	VARCHAR(64)	FOREIGN KEY REFERENCES local_pois(poi_id)	POI ID
day_index	INT	NOT NULL	第几天

字段名	类型	约束	说明
time_slot	VARCHAR(8)		时间段

5.2 索引设计

表名	索引名	字段	类型	说明
users	idx_open_id	open_id	UNIQUE	快速登录验证
conversations	idx_user_time	user_id, timestamp	BTREE	按用户和时间查询
itineraries	idx_user_created	user_id, created_at	BTREE	用户行程列表
local_pois	idx_city_type	city, type	BTREE	按城市和类型筛选
itinerary_poi	idx_itinerary_day	itinerary_id, day_index	BTREE	行程日活动查询

6 数据存储与访问策略

6.1 存储策略

- 所有表使用InnoDB引擎（MySQL）或默认事务存储（PostgreSQL）
- 敏感字段（如 `travel_preferences`）使用AES加密存储
- JSON字段用于灵活存储非结构化数据（如行程、偏好）

6.2 访问策略

- 使用ORM（SQLAlchemy）进行CRUD操作
- 高频查询（如热门POI）使用Redis缓存
- 读写分离：主库写，从库读（后续扩展）

7 安全性设计

7.1 用户权限

- 应用账号仅具备SELECT、INSERT、UPDATE权限，无DROP、ALTER权限
- 管理员账号独立，用于维护与备份

7.2 数据加密

- 用户偏好字段加密存储
- 传输层使用TLS/SSL

7.3 审计与日志

- 记录敏感操作日志（如用户删除行程）
- 数据库日志定期归档

8 备份与恢复

8.1 备份策略

- 每日全量备份（02:00 AM）
- 每6小时增量备份
- 备份文件保留30天

8.2 恢复测试

- 每月模拟恢复测试，确保备份有效性

9 维护与监控

- 使用Prometheus + Grafana监控数据库性能（QPS、连接数、慢查询）
- 定期执行 `OPTIMIZE TABLE`（MySQL）或 `VACUUM`（PostgreSQL）
- 错误日志告警接入系统告警平台

10 附录

10.1 示例数据

users 表示例：

```
{
  "user_id": "U001",
  "open_id": "wx_openid_123",
  "nickname": "旅行小张",
  "travel_preferences": {"budget": 1000, "style": "人文"}
}
```

itineraries 表示例：

```
{
  "itinerary_id": "I001",
  "user_id": "U001",
  "title": "北京三日游",
  "days": [
    {
      "date": "2025-11-01",
      "activities": [
        {"time": "09:00", "poi_id": "POI001", "type": "景点"}
      ]
    }
  ]
}
```

```
    }  
  ],  
  "total_budget": 950.0  
}
```